

BIBLIA

Uso moderno de IA, subagentes, SDD, AGENTS.md, GitHub, prompt engineering y harness engineering.

Autoría

Nicolás Ezequiel Melluso

nicolas.e.melluso@gmail.com

linkedin.com/in/nicolas-ezequiel-melluso

github.com/Nicolas-Melluso

Indice general

- 01****Uso Moderno de Inteligencia Artificial**

Como pasar de chatear con un modelo a trabajar con un sistema de pensamiento, ejecucion y verificacion

- 02****Uso Inteligente de Subagentes**

Criterios, patrones y cierre operativo para delegar mejor

- 03****SDD y Estructura de Soporte**

Specification-Driven Development aplicado a repositorios con IA

- 04****AGENTS.md, .github y Comandos Estilo Prompt**

Como construir un repositorio preparado para agentes, SDD, Copilot, workflows y prompts reutilizables

- 05****Prompt Engineering y Harness Engineering**

De prompts sueltos a sistemas versionados, evaluables y productivos

VOLUMEN 01

Uso Moderno de Inteligencia Artificial

Como pasar de chatear con un modelo a trabajar con un sistema de pensamiento, ejecucion y verificacion

La idea central

Usar inteligencia artificial de forma moderna no es abrir un chat, pedir "haceme esto" y aceptar la primera respuesta. Eso fue la primera etapa. La etapa moderna es tratar a la IA como una capa de trabajo: una combinacion de asistente, par tecnico, investigador, ejecutor, revisor y sistema de memoria. La diferencia no esta en escribir prompts mas largos. Esta en disenar un flujo donde cada interaccion deja contexto, artefactos, pruebas y decisiones reutilizables.

La forma inmadura de usar IA es conversacional y descartable: se pregunta algo, se obtiene una respuesta, se copia una parte y se sigue. La forma madura es operacional: se define el objetivo, se entrega contexto, se explicitan restricciones, se divide el trabajo, se verifica el resultado y se guarda lo aprendido. El valor aparece cuando la IA deja de ser una maquina de texto y empieza a funcionar como una extension del proceso de desarrollo, investigacion o produccion.

Este tomo propone un modelo de trabajo simple: pensar en la IA como un sistema con cuatro capas.

1. Capa de contexto: lo que la IA debe saber antes de actuar.
2. Capa de tarea: lo que debe producir ahora.
3. Capa de verificacion: como se sabe que el resultado sirve.
4. Capa de memoria: donde queda registrado para no repetir el mismo razonamiento.

Cuando esas cuatro capas existen, la IA puede ayudar en trabajos serios: escribir especificaciones, revisar codigo, comparar alternativas, generar documentacion, ejecutar pruebas, detectar riesgos, preparar presentaciones, entrenar a una persona o acelerar decisiones tecnicas. Cuando faltan, la IA se vuelve brillante por momentos y peligrosa en silencio.

Lo que cambio en la practica

El cambio importante no es solo que los modelos sean mejores. El cambio es que ahora se puede trabajar con agentes conectados a herramientas: editor, terminal, navegador, repositorio, issue tracker, documentacion, bases de datos locales, tests, linters, emuladores y pipelines. Antes el modelo respondia desde afuera del trabajo. Ahora puede participar dentro del trabajo.

Eso obliga a cambiar la forma de pedir. Un pedido moderno no dice solo "explicame X". Dice: "lee estos archivos, identifica el comportamiento actual, propone un cambio acotado, implementalo, corre estas pruebas y dejame un resumen con riesgos". La IA deja de ser un oraculo y pasa a ser un operador bajo contrato.

Tambien cambia el rol humano. El humano ya no gana por tipear todo manualmente. Gana por definir buen contexto, buenas restricciones, buenos criterios de aceptacion y buenos mecanismos de verificacion. La IA puede producir mucho, pero no sabe sola que tradeoff conviene para tu negocio, que riesgo legal aceptar, que deuda tecnica tolerar o que experiencia de usuario queres defender.

La pregunta moderna no es "que prompt uso". La pregunta moderna es "que sistema de trabajo hace que los resultados sean mejores cada semana".

El ciclo de trabajo recomendado

Un flujo robusto con IA puede verse así:

```
Intencion -> Contexto -> Plan -> Ejecucion -> Verificacion -> Registro -> Siguiete iteracion
```

La intencion define el resultado deseado. El contexto reduce ambigüedad. El plan evita trabajo impulsivo. La ejecucion produce artefactos concretos. La verificacion separa lo convincente de lo correcto. El registro evita que el conocimiento se pierda en el chat. La siguiente iteracion convierte el trabajo en aprendizaje acumulado.

Un ejemplo simple:

Intencion:

Quiero agregar autenticacion por magic link.

Contexto:

Repo Node/TypeScript, PostgreSQL, arquitectura por servicios, tests con Vitest.

Plan:

1. Ubicar auth actual.
2. Agregar tabla de tokens.
3. Implementar servicio.
4. Agregar pruebas unitarias e integracion.
5. Documentar variables de entorno.

Verificacion:

```
npm test
npm run typecheck
prueba manual del flujo login -> link -> sesion
```

Registro:

ADR corta sobre por que magic link y no password.
Spec de comportamiento esperado.
Checklist de rollback.

Ese flujo no depende de una herramienta especifica. Sirve con Codex, Copilot, Cursor, Claude Code, Gemini CLI o un agente propio. La madurez esta en el proceso.

La unidad minima de contexto

La IA trabaja mejor cuando recibe contexto empaquetado, no una nube de informacion. La unidad minima de contexto para una tarea seria deberia incluir:

Elemento	Para que sirve	Ejemplo
Objetivo	Evita que optimice otra cosa	"Reducir errores de checkout abandonado"
Estado actual	Le da punto de partida	"El retorno de Stripe vuelve a <code>/checkout/success</code> "
Restricciones	Acota soluciones	"No tocar precios ni migrar proveedor"
Archivos relevantes	Reduce exploracion ciega	<code>src/server.js</code> , <code>public/app.js</code>
Criterios de aceptacion	Define cierre	"Si vuelve de pago, muestra estado comprado"
Verificacion	Obliga a probar	"Smoke test local y test unitario"
Riesgos	Hace visible lo delicado	"No filtrar secretos en logs"

Sin esa unidad minima, el modelo completa huecos con supuestos. A veces acierta. En sistemas reales, a veces rompe cosas por seguir una logica que parecia razonable pero no pertenecia al proyecto.

Buenas tareas para IA

La IA es especialmente fuerte cuando puede operar sobre informacion explicita y cuando el resultado puede verificarse. Algunos usos de alto valor:

1. Resumir y mapear codigo existente.
2. Convertir una idea en una especificacion revisable.
3. Generar casos de prueba desde criterios de aceptacion.
4. Detectar inconsistencias entre docs, codigo y tests.
5. Refactorizar una zona acotada con suite de pruebas.
6. Preparar scripts de migracion o validacion.
7. Crear runbooks para operaciones repetibles.
8. Revisar PRs con reglas concretas.
9. Convertir conversaciones en tareas accionables.
10. Crear material de entrenamiento para otra persona.

La IA es menos confiable cuando se le pide decidir sin datos, inventar politicas de negocio, tocar muchas zonas del sistema a la vez, cambiar infraestructura sin permisos o producir contenido "definitivo" sin revision humana. No significa que no pueda ayudar. Significa que hay que darle un marco de control mas fuerte.

El prompt moderno

Un prompt moderno tiene forma de brief operativo. No necesita ser poetico ni enorme. Necesita eliminar ambigüedad.

Objetivo:

Quiero que conviertas esta idea en una especificacion SDD lista para implementar.

Contexto:

El producto es una app B2B para gestionar reclamos. El repo usa Node.js, TypeScript, PostgreSQL y GitHub Actions. Queremos mantener el alcance chico.

Entrada:

Idea: "permitir que un operador reasigne un reclamo a otro equipo".

Restricciones:

- No diseñar una pantalla completa todavía.
- No asumir roles nuevos si no son necesarios.
- Separar reglas de negocio de UI.
- Incluir riesgos y preguntas abiertas.

Salida:

1. Resumen del problema.
2. Requisitos funcionales.
3. Requisitos no funcionales.
4. Criterios de aceptacion.
5. Casos borde.
6. Plan de implementacion en 3 slices.
7. Pruebas recomendadas.

Calidad:

Si falta informacion, marca como pregunta abierta. No inventes datos de negocio.

Ese formato hace tres cosas: le da direccion, le da limites y define como evaluar la respuesta. No busca "inspirar" al modelo. Busca contratarlo para una tarea.

De chat a repositorio

El salto mas importante es mover conocimiento desde el chat hacia el repositorio. El chat es fragil: se pierde, se contradice, no versiona bien, no se revisa en PR y no corre en CI. El repositorio, en cambio, puede guardar instrucciones, specs, decisiones, prompts, tests y workflows.

Una organizacion moderna suele separar:

Artefacto	Audiencia	Funcion
<code>README.md</code>	Humanos nuevos	Presentar el proyecto y como empezar
<code>AGENTS.md</code>	Agentes de codigo	Reglas operativas, comandos, estilo y verificacion
<code>.github/copilot-instructions.md</code>	Copilot	Instrucciones generales para respuestas en el repo
<code>.github/instructions/*.instructions.md</code>	Copilot por path	Reglas especificas por zona del codigo
<code>.github/prompts/*.prompt.md</code>	Humanos y asistentes	Comandos reutilizables estilo prompt
<code>.github/orquestador/sdd/*</code>	Equipo y agentes	Specs, decisiones, tareas, trazabilidad
<code>.github/workflows/*</code>	CI/CD	Automatizacion ejecutable

La regla practica: lo que se repite debe vivir como archivo. Si cada vez que pedis ayuda tenes que explicar los mismos comandos, el mismo estilo, las mismas restricciones y los mismos criterios de prueba, eso no es prompt engineering. Es deuda de contexto.

Roles de IA en un equipo chico

Aunque una sola herramienta parezca "un asistente", conviene pensar en roles:

Rol	Que hace	Buen output
Investigador	Lee, compara, resume, encuentra patrones	Mapa de archivos, riesgos, preguntas
Planner	Divide el trabajo	Plan por slices, dependencias, verificacion
Implementador	Cambia archivos	Patch acotado, tests, resumen
Revisor	Busca errores	Findings con archivo y linea
Documentador	Convierte trabajo en conocimiento	README, ADR, runbook
Evaluador	Prueba comportamiento	Reporte de comandos y resultados

Los subagentes formalizan esa separacion, pero el modelo mental sirve incluso con un solo chat. Cuando una misma conversacion intenta hacer todo al mismo tiempo, se vuelve confusa. Cuando cada rol tiene una salida concreta, el trabajo se vuelve controlable.

La verificacion no es opcional

La IA puede producir texto convincente y codigo que parece correcto. La unica defensa seria es verificar. La verificacion puede ser automatica o humana, pero debe existir.

Ejemplos de verificacion:

1. Tests unitarios y de integracion.
2. Typecheck, lint y build.
3. Smoke test manual documentado.
4. Comparacion contra criterios de aceptacion.
5. Revision de diffs archivo por archivo.
6. Prueba con datos reales o fixtures.
7. Evaluacion con golden cases para prompts.
8. Checklist de seguridad y permisos.

La frase "se ve bien" no alcanza. Un flujo moderno pide que la IA diga que ejecuto, que no pudo ejecutar, que cambio, que falta y que riesgo queda.

Memoria util, no memoria infinita

La memoria sirve cuando reduce repeticion y mejora consistencia. No sirve cuando se convierte en un basural de notas largas. Una memoria util tiene tres propiedades:

1. Es recuperable: esta en una ruta conocida.
2. Es accionable: contiene decisiones, comandos, convenciones o errores aprendidos.
3. Es verificable: no reemplaza al estado actual del repo cuando este puede haber cambiado.

Ejemplos de memoria buena:

- El repo usa ``.github/orquestador`` como carpeta de contexto.
- Los workflows son la unica capa ejecutable; el catalogo solo documenta.
- Antes de cerrar cambios de runtime correr ``npm test`` y ``npm run build``.
- En Windows, verificar locks antes de renombrar carpetas.

Ejemplos de memoria mala:

- El proyecto es importante.
- A veces falla.
- Usar buenas practicas.

La memoria moderna no guarda sentimientos. Guarda operaciones.

Plan de adopcion en 30 dias

Semana 1: ordenar el contexto

Crear `AGENTS.md`, documentar comandos reales, listar restricciones y definir donde vive el SDD. El objetivo no es cubrir todo. Es que un agente pueda entrar al repo y no perder media hora adivinando.

Entregables:

1. `AGENTS.md` inicial.
2. `README.md` actualizado.
3. `.github/orquestador/context/product.md`.
4. `.github/orquestador/context/architecture.md`.
5. Lista de comandos de verificación.

Semana 2: trabajar por especificaciones

Elegir una feature chica y escribir una spec antes de implementar. Incluir criterios de aceptación, casos borde, no objetivos y pruebas. Después pedir a la IA que implemente solo un slice.

Entregables:

1. Primera spec SDD.
2. Plan por slices.
3. ADR si hay una decisión técnica relevante.
4. Tests asociados.

Semana 3: introducir prompts reutilizables

Crear prompts para tareas repetidas: planificar feature, revisar PR, generar tests, escribir ADR, preparar runbook. Guardarlos en `.github/prompts` o en la carpeta de orquestación elegida.

Entregables:

1. `plan-feature.prompt.md`.
2. `review-pr.prompt.md`.
3. `write-adr.prompt.md`.
4. `generate-tests.prompt.md`.

Semana 4: medir calidad

Agregar harnesses o evaluaciones simples. No hace falta montar una plataforma enorme. Empezar con fixtures, casos esperados y un script que compare salidas.

Entregables:

1. Carpeta `evals/`.
2. Fixtures de entrada.
3. Rubrica de evaluación.
4. Script local o workflow de validación.

Anti-patrones comunes

Anti-patron	Sintoma	Correccion
Prompt gigante para todo	El modelo ignora partes	Dividir en instrucciones persistentes y tareas concretas
Sin verificacion	Outputs lindos pero fraguados	Definir comandos y criterios de aceptacion
Contexto solo en chat	Se repite todo cada sesion	Mover reglas al repo
Agente con permisos amplios	Riesgo de cambios destructivos	Ownership y permisos minimos
Todo en un solo subagente	Paralelismo falso	Separar exploracion, implementacion y revision
Docs que no corren	Buenas intenciones sin efecto	Conectar docs con workflows y checklists

Checklist para usar IA de forma moderna

Antes de pedir:

1. Tengo claro el resultado final.
2. Puedo nombrar los archivos o dominios relevantes.
3. Se que no quiero que toque.
4. Tengo una forma de verificar.
5. Puedo aceptar un primer slice en vez de todo el sistema.

Durante el trabajo:

1. Pido planes cortos para tareas riesgosas.
2. Divido investigacion, edicion y revision.
3. Mantengo ownership de archivos.
4. Leo los diffs antes de cerrar.
5. Registro decisiones nuevas.

Al cerrar:

1. Se que cambio.
2. Se que pruebas pasaron.
3. Se que pruebas no se corrieron.
4. Se que riesgos quedan.
5. El conocimiento reusable quedo en archivos.

Cierre

La IA moderna no reemplaza el oficio. Lo amplifica cuando el oficio esta ordenado. El usuario que mas obtiene no es el que sabe el truco secreto del prompt, sino el que sabe convertir trabajo ambiguo en unidades verificables.

La regla final es simple: si el output importa, tratá a la IA como parte de un sistema de produccion. Dale contexto, limites, herramientas, pruebas y memoria. Lo demas es solo chat.

VOLUMEN 02

Uso Inteligente de Subagentes

Crterios, patrones y cierre operativo para delegar mejor

Para qué sirve este volumen

Trabajar con agentes y subagentes modernos no consiste en repartir tareas al azar. La diferencia entre una delegación útil y una pérdida de tiempo suele estar en tres cosas: el tamaño del problema, la calidad del encuadre y el nivel de control sobre el resultado. Cuando se hace bien, el trabajo se acelera, el contexto se usa mejor y el hilo principal queda libre para decisiones que realmente requieren criterio humano o arquitectónico.

Este documento propone una forma práctica de usar subagentes en proyectos técnicos. No intenta vender magia ni automatización total. El objetivo es más modesto y más útil: que puedas dividir trabajo con criterio, pedirle a cada subagente lo justo, conservar trazabilidad y cerrar cada paso con evidencia.

La idea central es esta: un subagente no es un reemplazo de pensamiento. Es una unidad de ejecución acotada que trabaja con contexto mínimo, propiedad clara y una salida verificable. Si falta alguna de esas piezas, conviene no delegar todavía.

Cuándo delegar

No todo merece un subagente. Delegar demasiado pronto genera ruido, duplicación y respuestas que parecen correctas pero no resuelven el problema. Delegar demasiado tarde te deja haciendo a mano tareas repetitivas que podrían haberse resuelto en paralelo.

Una buena regla práctica es delegar cuando se cumplen varias de estas condiciones:

- La tarea está bien delimitada.
- El resultado esperado se puede verificar.
- La tarea no necesita decisiones cruzadas con otras partes del sistema.
- Hay trabajo repetitivo, exploratorio o mecánico suficiente como para justificar el costo de coordinar.
- El contexto necesario cabe en unas pocas instrucciones y archivos concretos.
- El riesgo de que el subagente toque algo sensible está contenido por ownership o por read-only.

Ejemplos buenos de delegación:

- Buscar dónde está implementado un comportamiento.
- Mapear flujo de datos entre módulos.
- Identificar tests existentes para una zona del código.
- Redactar un borrador de documento a partir de notas o una estructura previa.
- Ejecutar una validación puntual sobre una carpeta acotada.
- Probar hipótesis técnicas que no requieren editar muchas piezas a la vez.

Ejemplos malos de delegación:

- "Arreglá todo el sistema".
- "Rehacé la arquitectura".
- "Entrale al repo y mejorá lo que veas".
- "Tomá decisiones de producto sin criterio de aceptación".

- “Optimizá rendimiento” sin un punto de partida, una métrica y un alcance.

Si no podés decir qué archivo o qué salida querés, probablemente no estás listo para delegar.

Explorer y worker

La separación más útil en un flujo con subagentes es la de **explorer** y **worker**.

Explorer

El explorer sirve para entender. Su trabajo es leer, mapear, comparar y devolver información condensada. No debería hacer cambios salvo que la tarea sea explícitamente de exploración con anotación local, y aun así conviene mantenerlo read-only por defecto.

Usos típicos:

- Encontrar implementaciones.
- Seguir referencias.
- Resumir patrones existentes.
- Detectar tests, scripts o configuraciones relevantes.
- Comparar alternativas sin tocar el código.

Qué le pedís:

- que cite archivos concretos;
- que resuma con bullets cortos;
- que no proponga soluciones innecesarias;
- que identifique incertidumbres;
- que marque qué no pudo confirmar.

El explorer es especialmente valioso cuando no sabés todavía cuál es la superficie real del cambio. Antes de editar, primero entendé el mapa.

Worker

El worker sirve para hacer. Recibe un objetivo acotado, una zona de edición clara y criterios de salida verificables. Puede editar archivos, correr comandos o preparar un parche, pero siempre dentro de un ownership explícito.

Usos típicos:

- Implementar una función puntual.
- Crear un script de validación.
- Ajustar un archivo de documentación.
- Probar una hipótesis en una rama de trabajo o sobre un subconjunto de archivos.
- Preparar fixtures o datos de prueba.

Qué le pedís:

- que toque solo archivos autorizados;
- que no revierta cambios ajenos;
- que explique qué cambió;
- que valide con comandos concretos;
- que deje el trabajo listo para revisión.

La regla general es simple: explorer para reducir incertidumbre, worker para ejecutar una tarea ya entendida.

Contexto mínimo

Uno de los errores más comunes al delegar es pasar demasiado contexto. Darle a un subagente "todo lo que hay" parece seguro, pero suele degradar la calidad. A mayor contexto, más ruido, más costo y más chances de que el agente mezcle señales irrelevantes.

El buen contexto mínimo responde estas preguntas:

1. Qué querés lograr.
2. Qué parte del sistema puede tocar.
3. Qué archivos o rutas son relevantes.
4. Qué no debe tocar.
5. Cómo sabés si terminó bien.

Un encuadre útil puede tener esta forma:

- Objetivo: una frase concreta.
- Alcance: archivos y carpetas.
- Restricciones: no tocar X, no modificar Y, no cambiar comportamiento fuera de Z.
- Salida esperada: resumen, parche, lista de hallazgos o script.
- Verificación: tests, comandos o revisión manual.

No hace falta explicar toda la historia del proyecto. Hace falta explicar la porción que el subagente necesita para actuar sin improvisar.

Qué incluir

- El problema exacto.
- El formato de la salida.
- Los archivos que marcan ownership.
- Los comandos de validación.
- Los criterios de aceptación.

Qué evitar

- Historias largas sin relevancia operativa.
- Opiniones contradictorias.

- Pistas viejas que ya no aplican.
- Capturas mentales que el agente no puede verificar.
- Instrucciones abiertas como “mejoralo bastante”.

Si el subagente necesita una aclaración clave para no errar, mejor frenar y redefinir. Si solo necesita detalles ruidosos, no los pases.

Ownership de archivos

El ownership es lo que evita que varios agentes pisen el mismo terreno. Cada tarea debe tener una frontera clara: qué archivos puede editar un worker, qué archivos solo puede leer, y qué partes del sistema están fuera de juego.

Esto no es solo higiene. Es una forma de preservar integridad mientras se trabaja en paralelo.

Una asignación bien hecha incluye:

- archivo o carpeta autorizada;
- tipo de permiso: lectura, edición o solo inspección;
- límites de impacto;
- criterio para considerar invasivo un cambio;
- confirmación de que no debe revertir trabajo ajeno.

Ejemplo de ownership bueno:

- Worker A: `src/docs/intro.md` y `src/docs/glosario.md`, solo edición de contenido.
- Worker B: `scripts/validate-docs.mjs`, solo este archivo y su test asociado.
- Explorer C: cualquier archivo bajo `src/`, pero sin editar.

Ejemplo de ownership malo:

- “Editá lo que haga falta”.
- “Acomodá todo el módulo”.
- “Mirá si encontrás algo mejor”.

Cuando el ownership está claro, la revisión también mejora. Sabés qué cambió, por qué cambió y qué queda fuera del scope.

Paralelismo

Los subagentes brillan cuando podés dividir trabajo independiente. El paralelismo no significa hacer más cosas al mismo tiempo por ansiedad, sino separar tareas que no se bloquean entre sí.

Hay tres niveles útiles:

Paralelismo de exploración

Varios explorers buscan información distinta en paralelo.

Ejemplo:

- uno localiza la implementación;
- otro identifica tests;
- otro resume patrones similares;
- otro busca riesgos o dependencias.

Esto sirve mucho al inicio de una tarea grande. En vez de leer todo secuencialmente, obtenés un mapa más rápido.

Paralelismo de ejecución

Varios workers hacen cambios en áreas distintas, siempre que los archivos no se solapen.

Ejemplo:

- uno edita documentación;
- otro ajusta una validación;
- otro prepara ejemplos o fixtures.

Esto requiere disciplina. Si dos workers comparten archivos, el supuesto de paralelismo se rompe y empezás a competir con merges o reescrituras.

Paralelismo de verificación

Un subagente implementa y otro verifica desde afuera.

Ejemplo:

- worker A cambia una función;
- worker B revisa que los tests relevantes existan y que el cambio no rompa convenciones;
- el hilo principal integra el resultado.

Este patrón es útil para separar producción de evidencia. La verificación independiente reduce sesgo de confirmación.

Cuándo no paralelizar

No conviene paralelizar cuando:

- la decisión de una tarea depende del resultado de otra;
- el mismo archivo será editado por varios agentes;
- la arquitectura todavía está en discusión;
- el costo de coordinar supera el ahorro;
- un error podría contaminar varias piezas a la vez.

Paralelizar no es un objetivo en sí. Es una herramienta cuando la independencia existe de verdad.

Anti-patrones

Los anti-patrones suelen verse como productividad al principio y como deuda operativa después.

1. Delegación vaga

Pedís algo demasiado amplio y obtenés algo demasiado genérico. El agente rellena huecos con suposiciones.

Señal típica: la respuesta suena prolija pero no aterriza en archivos ni decisiones concretas.

2. Context dumping

Le pasás todo el repo, todo el chat, todas las notas. El agente pierde foco.

Señal típica: respuestas largas con fragmentos relevantes mezclados con ruido.

3. Ownership difuso

Nadie sabe quién puede tocar qué. Aparecen ediciones cruzadas, pisadas y confusión.

Señal típica: "yo pensé que esa carpeta era libre".

4. Subagentes que resuelven diseño

El subagente empieza a inventar estrategia cuando solo tenía que ejecutar una slice concreta.

Señal típica: propone reestructurar todo antes de terminar el cambio pedido.

5. Paralelismo falso

Se lanzan varias tareas "en paralelo" que en realidad compiten por la misma zona.

Señal típica: conflicto de archivos, resultados inconsistentes o necesidad de rehacer trabajo.

6. Cierre sin evidencia

El subagente dice que terminó, pero no muestra cómo se valida.

Señal típica: no hay comando, no hay archivo, no hay criterio de aceptación.

7. Mezclar lectura con edición sin control

Un agente explora y además toca cosas fuera de scope porque "ya que estaba".

Señal típica: cambios colaterales no pedidos.

La regla de oro es dura pero simple: si una tarea no se puede revisar con claridad, probablemente fue delegada mal.

Matriz de modelos y esfuerzo

No todos los subagentes necesitan el mismo tipo de modelo ni el mismo nivel de razonamiento. Conviene pensar en una matriz práctica: complejidad de la tarea por esfuerzo requerido.

La idea no es memorizar nombres, sino usar una combinación razonable según el trabajo.

Tareas mecánicas

- Ejemplos: inspección de archivos, búsqueda, extracción de patrones, validaciones simples, formateo.
- Modelo sugerido: uno liviano o rápido.
- Esfuerzo: bajo.
- Objetivo: velocidad y bajo costo.

Tareas de síntesis

- Ejemplos: resumir hallazgos, comparar opciones, condensar documentación, mapear flujo.
- Modelo sugerido: liviano o medio, según la densidad del material.
- Esfuerzo: bajo a medio.
- Objetivo: ordenar información sin sobredimensionar la respuesta.

Tareas de implementación acotada

- Ejemplos: editar un archivo, crear un script, ajustar una prueba.
- Modelo sugerido: medio.
- Esfuerzo: medio.
- Objetivo: buen criterio técnico sin costo excesivo.

Tareas de debugging complejo

- Ejemplos: bugs con múltiples causas, integración cruzada, fallas intermitentes.
- Modelo sugerido: uno más fuerte.
- Esfuerzo: medio a alto.
- Objetivo: más capacidad de razonamiento, menos atajos.

Tareas de revisión final

- Ejemplos: revisar un parche importante, detectar regresiones, cuestionar supuestos.
- Modelo sugerido: más fuerte o al menos distinto al que implementó.
- Esfuerzo: medio a alto.
- Objetivo: independencia y mirada crítica.

Regla práctica

- Si la tarea es repetitiva, no gastes un modelo pesado.
- Si la tarea depende de criterios finos o cruza varias piezas, subí el nivel.
- Si el resultado va a decidir una entrega importante, agregá revisión independiente.

El esfuerzo no debe ser "siempre alto". Tiene que acompañar el riesgo y la ambigüedad.

Ejemplos de prompts de delegación

Los mejores prompts de subagente no parecen pedidos abiertos. Parecen tickets bien escritos.

Exploración de implementación

Objetivo: encontrar dónde se implementa el flujo de guardado automático.
Rol: explorer.
Alcance: solo lectura sobre `src/` y `tests/`.
Entrega: lista de archivos relevantes, resumen breve del flujo y riesgos.
No hagas cambios.
Si algo no está claro, marca la incertidumbre.

Mapeo de tests

Objetivo: identificar tests existentes para la validación de rutas.
Rol: explorer.
Alcance: buscar en `tests/`, `spec/` y scripts de CI.
Entrega: tabla simple con archivo, propósito y cobertura.
No edites nada.

Implementación acotada

Objetivo: agregar un helper para normalizar títulos en `src/utils/title.js`.
Rol: worker.
Ownership exclusivo: solo `src/utils/title.js`.
Restricciones: no tocar otros archivos, no cambiar interfaces públicas.
Entrega: código final, explicación breve y comando de verificación.

Documento o borrador

Objetivo: redactar un borrador técnico sobre uso de subagentes.
Rol: worker.
Ownership: solo `src/02-subagentes-inteligentes.md`.
Estilo: claro, serio, accionable, en español rioplatense neutro.
Incluí criterios, anti-patronos, ejemplos y checklist de cierre.
No generes archivos extra.

Verificación independiente

Objetivo: revisar el cambio y buscar errores lógicos o scope creep.
Rol: explorer.
Alcance: leer el parche y los archivos tocados.
Entrega: hallazgos concretos, dudas abiertas y sugerencias de validación.
No modifiques archivos.

Fijate que en todos los casos el prompt define rol, alcance, entrega y límites. Eso reduce ambigüedad y mejora la calidad de salida.

Checklist de cierre

Una delegación no termina cuando el subagente responde. Termina cuando el resultado quedó verificado y el hilo principal puede seguir sin arrastrar dudas.

Checklist de cierre para usar siempre:

- El objetivo quedó cumplido en una frase verificable.
- El subagente trabajó dentro del ownership autorizado.
- No hubo cambios fuera de scope.
- Los archivos tocados quedaron identificados.
- La salida es reproducible o revisable.
- Si hubo cambios de código, existe comando de validación.
- Si hubo cambios de documento, el contenido cumple la estructura pedida.
- Las incertidumbres quedaron explícitas.
- No hay conflictos con trabajo paralelo.
- El resultado final fue leído por el hilo principal antes de considerarlo cerrado.

Una versión más estricta para tareas sensibles:

- Revisé el diff.
- Verifiqué que no se tocaron archivos ajenos.
- Corrí el comando de validación pertinente.
- Confirmé que no hay supuestos ocultos.
- Registré qué queda pendiente, si algo quedó afuera.

Proceso recomendado

Un flujo robusto para usar subagentes suele verse así:

1. Definís la tarea y su criterio de salida.
2. Separás lo que es exploración de lo que es ejecución.
3. Asignás ownership explícito.
4. Elegís el nivel de modelo y esfuerzo según ambigüedad y riesgo.
5. Corrés tareas en paralelo solo si no se pisan.
6. Consolidás resultados en el hilo principal.
7. Verificás el cierre con evidencia.

Si la tarea es grande, este flujo se puede repetir por capas: primero explorers, después workers, después revisión independiente.

Señales de que vas bien

Vas bien cuando:

- el subagente responde con menos texto pero más precisión;
- cada salida menciona archivos o comandos concretos;
- el hilo principal decide más rápido porque recibió información mejor filtrada;
- el paralelismo reduce tiempo sin aumentar retrabajo;
- los errores se detectan antes de integrar.

Vas mal cuando:

- tenés que reinterpretar todo lo que devolvieron;
- aparecen cambios sorpresa;
- el contexto crece sin control;
- la verificación se hace recién al final y con sorpresas;
- cada subagente necesita que le expliques de nuevo la misma base.

Cierre operativo

Usar subagentes con inteligencia no es una cuestión de cantidad, sino de forma. La calidad de la delegación depende más del recorte, la propiedad y la verificación que del número de agentes en juego.

Si necesitás explorar, usá `explorer`. Si necesitás ejecutar, usá `worker`. Si necesitás varias cosas a la vez, dividí el trabajo de forma que no se pisen. Y si no podés definir un `ownership` claro, todavía no conviene delegar.

La mejor práctica no es "hacer que la IA trabaje sola". Es armar una cadena de trabajo donde cada pieza tenga un rol acotado, una salida comprobable y un cierre explícito. Ahí es cuando los subagentes dejan de ser una promesa y pasan a ser una herramienta útil de verdad.

VOLUMEN 03

SDD y Estructura de Soporte

Specification-Driven Development aplicado a repositorios con IA

SDD, o *Specification-Driven Development*, es una forma de trabajar en la que la especificación no es un documento decorativo ni una nota aislada: es la pieza central del flujo de desarrollo. En lugar de arrancar por código suelto, arrancamos por una definición clara de lo que se quiere construir, por qué existe, cómo se va a validar y qué decisiones quedan registradas en el camino.

En un repositorio moderno, y más todavía cuando hay IA involucrada, SDD ordena el trabajo en capas. Primero se define el problema. Después se convierte ese problema en una especificación verificable. Luego se descompone en tareas. Recién ahí se implementa. Y al final se deja trazabilidad: qué se cambió, qué se descartó, qué se probó y qué quedó listo para operar.

Esto sirve especialmente cuando el equipo usa IA como copiloto, como generadora de borradores o como asistente de análisis. La IA puede acelerar muchísimo, pero también puede inventar supuestos, mezclar prioridades o producir código que "funciona" sin respetar el contexto. SDD le pone límites útiles a eso. La IA no adivina el objetivo: lo lee. No improvisa criterios de aceptación: los sigue. No reemplaza decisiones: las documenta o las propone para aprobación.

Qué resuelve SDD

SDD resuelve un problema muy concreto: la distancia entre la intención y la implementación. En repositorios grandes, esa distancia suele crecer en silencio. Un issue dice una cosa, un PR resuelve otra, el código termina haciendo algo más, y nadie sabe cuál era la versión correcta de la idea.

Con SDD, esa cadena queda explícita:

- **requirements** : qué necesidad existe y a quién le importa.
- **specs** : cómo se describe el comportamiento esperado.
- **decisions** : qué alternativas se evaluaron y cuál se eligió.
- **tasks** : qué pasos concretos hay que ejecutar.
- **acceptance criteria** : cómo se sabe que quedó bien.
- **traces** : qué evidencia conecta la spec con el código y las pruebas.
- **runbooks** : cómo operarlo o recuperarlo cuando falla.
- **ADRs** : decisiones de arquitectura con contexto y consecuencias.
- **evaluation notes** : resultados de validaciones, pruebas asistidas o análisis comparativos.

La ventaja no es solo documental. También mejora la ejecución. Cuando la estructura es buena, la IA genera menos basura, el equipo discute mejor y los cambios son más fáciles de revisar.

Principio base

La regla principal de SDD es simple: cada pieza debe tener un dueño semántico.

- El requerimiento explica el problema.
- La spec define el comportamiento.
- La decisión explica el trade-off.
- La tarea ordena el trabajo.

- La aceptación cierra el alcance.
- La traza permite seguir el hilo.
- El runbook prepara la operación.

Si todo está mezclado en un solo archivo, el sistema se vuelve frágil. Si está demasiado atomizado sin criterio, también. SDD busca un equilibrio: separar lo suficiente para que cada cosa tenga función propia, pero no tanto como para que entender una feature requiera abrir veinte archivos sin relación.

Carpeta recomendada

La convención sugerida para este volumen es concentrar el material de SDD alrededor de [.github/orquestador/sdd](https://github.com/orquestador/sdd). Esa carpeta funciona como núcleo de gobernanza y ejecución para specs, decisiones y trabajo asistido por IA.

Una estructura posible es esta:

```
.github/orquestador/sdd/  
  README.md  
  index.md  
  requirements/  
    000-template.md  
    <area>--<id>.md  
  specs/  
    000-template.md  
    <feature>--<id>.md  
  decisions/  
    000-template-adr.md  
    <adr>--<id>.md  
  tasks/  
    000-template.md  
    <issue>--<id>.md  
  acceptance/  
    000-template.md  
    <feature>--<id>.md  
  traces/  
    000-template.md  
    <feature>--<id>.md  
  runbooks/  
    000-template.md  
    <system>--<id>.md  
  evaluations/  
    000-template.md  
    <feature>--<id>.md  
  examples/  
    sample-issue.md  
    sample-spec.md
```

No hace falta que todas las carpetas existan desde el día uno. Lo importante es que la arquitectura esté pensada para escalar sin perder legibilidad.

Qué va en cada carpeta

README.md

Es la puerta de entrada. Debe decir qué es SDD en este repositorio, cuál es el objetivo de la carpeta y cómo se usa. No debería contener la teoría completa, sino una guía corta para navegar el sistema.

Contenido esperado:

- propósito de la carpeta;
- convención de nombres;
- orden recomendado de lectura;
- cómo crear una nueva spec o una nueva decision;
- relación con issues, PRs y documentación general.

index.md

Es el mapa de navegación. Lista las specs activas, los ADRs relevantes, las evaluaciones más recientes y los runbooks críticos. También puede incluir estado: borrador, en revisión, aprobado, implementado, obsoleto.

Contenido esperado:

- catálogo de artefactos vivos;
- links cruzados;
- estado por documento;
- fecha de última actualización;
- responsable o área.

requirements/

Acá viven los requerimientos. No son tickets técnicos todavía. Son necesidades, dolores, objetivos o restricciones del negocio, producto u operación.

Un requerimiento bien escrito responde:

- qué problema existe;
- para quién;
- qué pasa si no se resuelve;
- qué restricciones hay;
- qué señales indicarían éxito.

Ejemplo breve:

REQ-014: reducir errores en la alta de clientes

Problema: hoy los formularios permiten guardar datos incompletos y eso genera retrabajo.

Objetivo: bloquear altas inválidas antes de persistir.

Restricciones: no romper el flujo actual de edición.

Impacto: menos soporte manual y menos errores en reportes.

specs/

La spec describe el comportamiento. Ya no habla solo del problema, sino del sistema esperado. Tiene que ser verificable. Una buena spec deja en claro entradas, salidas, reglas, estados, errores, permisos y escenarios límite.

Contenido esperado:

- contexto;
- alcance;
- comportamiento principal;
- casos borde;
- flujos alternativos;
- dependencias;
- criterios de aceptación;
- riesgos;
- supuestos explícitos.

La spec no debería escribir código, pero sí puede incluir pseudo-reglas, ejemplos de payload, tablas de estados o secuencias.

decisions/

Acá viven los ADRs y cualquier decisión estructural que valga la pena recordar. Si la spec dice *qué* hacer, la decisión explica *por qué* se eligió ese camino y no otro.

Contenido esperado:

- contexto;
- opciones consideradas;
- decisión tomada;
- consecuencias;
- trade-offs;
- fecha;
- autor o equipo.

Ejemplo de decisión:

```
# ADR-007: validar en servidor y no solo en cliente
```

Se elige validar en backend como fuente de verdad.

Razón: el cliente puede quedar desactualizado y no es confiable como única barrera.

Consecuencia: se duplica parte de la lógica, pero se reduce el riesgo operativo.

tasks/

Las tasks descomponen la implementación en pasos chicos y verificables. Son el puente entre la spec y el código. Una task buena no describe una aspiración vaga, sino una acción concreta.

Contenido esperado:

- objetivo de la task;
- dependencias;
- archivos o módulos involucrados;
- criterio de terminado;
- evidencia esperada.

Una task mal escrita dice: "hacer la feature". Una task útil dice: "agregar validación del campo X en el endpoint Y y cubrirlo con test de integración".

acceptance/

Esta carpeta contiene los criterios de aceptación. Su función es cerrar el contrato entre lo pedido y lo entregado. Tiene que ser lo bastante concreta para que un reviewer o una IA puedan chequearla sin inventar interpretación.

Contenido esperado:

- condiciones de pase/falla;
- escenarios nominales;
- errores esperados;
- comportamiento observable;
- requisitos no funcionales si aplican.

traces/

Las traces conectan el requerimiento, la spec, las tareas y el resultado final. Sirven para saber de dónde salió cada decisión y qué evidencia la respalda.

Contenido esperado:

- vínculo `REQ -> SPEC -> TASK -> PR -> TEST` ;
- resumen de cobertura;
- referencias a commits, issues o PRs;
- notas sobre lo que quedó fuera.

runbooks/

Los runbooks documentan cómo operar o recuperar un flujo cuando algo falla. Son especialmente útiles cuando una spec termina en una feature con impacto real: pagos, autenticación, jobs, sincronizaciones, migraciones o procesos automáticos.

Contenido esperado:

- síntomas comunes;
- pasos de diagnóstico;
- pasos de recuperación;
- rollback si aplica;
- señales de alerta;
- contactos o dependencias.

evaluations/

Acá van las notas de evaluación: pruebas humanas, pruebas con IA, benchmarks internos, comparaciones antes/después, análisis de riesgo o revisiones de calidad.

Contenido esperado:

- qué se evaluó;
- con qué método;
- qué resultado dio;
- qué defectos aparecieron;
- qué decisión se tomó a partir de eso.

examples/

Sirve para bajar fricción. Tener plantillas y ejemplos concretos mejora mucho la adopción, sobre todo si el equipo recién empieza a usar SDD.

Contenido esperado:

- issue ejemplo;
- spec ejemplo;
- ADR ejemplo;
- task ejemplo;
- checklist ejemplo.

Ciclo de trabajo de una issue o feature

Un flujo SDD sano no arranca en el PR. Arranca antes, cuando todavía se puede corregir el rumbo barato.

1. Abrir el requerimiento

Primero se redacta el problema. No hace falta poesía. Hace falta precisión. El requerimiento debe explicar por qué existe la iniciativa y qué dolor resuelve.

2. Convertirlo en spec

Después se define el comportamiento esperado. Aquí se responde:

- qué entra;
- qué sale;
- qué reglas aplican;
- qué errores se permiten;
- qué no entra en esta versión.

3. Registrar decisiones

Si hay alternativas relevantes, se deja un ADR. Esto evita que, dentro de dos meses, alguien vuelva a discutir una decisión ya tomada como si nada hubiera pasado.

4. Dividir en tasks

La implementación se parte en tareas chicas. Cada task debería poder hacerse, revisarse y testearse de forma independiente.

5. Definir aceptación

Antes de tocar código, los criterios de aceptación ya tienen que estar escritos. Si se escriben después, suelen adaptarse al resultado y pierden valor.

6. Implementar con trazabilidad

Cada cambio debe dejar conexión con la spec. Eso puede ser mediante referencias en el PR, notas en la task o vínculos en la trace. Lo importante es que el hilo no se corte.

7. Evaluar

Al final se registra lo que pasó: pruebas realizadas, problemas detectados, coberturas faltantes, deuda pendiente y decisión final.

Un ejemplo de ciclo completo:

1. **REQ-014** detecta errores en la alta de clientes.
2. **SPEC-014** define validaciones, mensajes y estados.
3. **ADR-007** elige validar en backend.
4. **TASK-014.1** agrega validaciones.
5. **TASK-014.2** cubre tests de integración.
6. **AC-014** define 5 criterios de aceptación.
7. **TRACE-014** enlaza issue, PR y pruebas.
8. **EVAL-014** resume el resultado y puntos abiertos.

Plantillas cortas

Requerimiento

```
# REQ-XXX: título corto
```

```
Problema:
```

```
Objetivo:
```

```
Usuarios impactados:
```

```
Restricciones:
```

```
Riesgo de no hacerlo:
```

Spec

```
# SPEC-XXX: título corto
```

```
Contexto:
```

```
Alcance:
```

```
Fuera de alcance:
```

```
Comportamiento esperado:
```

```
Casos borde:
```

```
Criterios de aceptación:
```

ADR

```
# ADR-XXX: decisión corta
```

```
Contexto:
```

```
Opciones:
```

```
Decisión:
```

```
Consecuencias:
```

Task

```
# TASK-XXX: acción concreta
```

```
Qué hay que hacer:
```

```
Archivos probables:
```

```
Dependencias:
```

```
Cómo se valida:
```

Acceptance

```
# AC-XXX: feature o spec

- Dado ...
- Cuando ...
- Entonces ...
```

Trace

```
# TRACE-XXX: título corto

REQ:
SPEC:
TASKS:
PR:
TESTS:
NOTAS:
```

Runbook

```
# RUN-XXX: sistema o flujo

Síntoma:
Diagnóstico:
Recuperación:
Rollback:
```

Evaluation notes

```
# EVAL-XXX: título corto

Qué se probó:
Criterio:
Resultado:
Problemas:
Decisión:
```

Cómo usar SDD con IA

La IA cambia el valor de la documentación. Antes, un spec servía sobre todo para humanos. Ahora también sirve como contrato de contexto para asistentes.

Un buen flujo con IA es este:

- la IA lee el requerimiento y resume el problema;
- la IA propone una spec inicial;
- el humano valida alcance y prioridades;
- la IA descompone tasks y sugiere tests;
- el humano aprueba o corrige decisiones;
- la IA ayuda a implementar y a revisar trazabilidad;
- el humano verifica aceptación final.

Eso funciona mejor cuando el repositorio tiene artefactos estables y legibles. Si la spec está desordenada, la IA va a producir respuestas desordenadas. Si la aceptación es vaga, la IA va a completar los huecos con supuestos. Si no hay ADRs, las decisiones se pierden y el contexto se reinicia cada vez.

La clave es usar la IA para acelerar razonamiento, no para reemplazarlo. SDD hace que la IA trabaje sobre una base explícita. En vez de "haceme una feature", se le pide "tomando esta spec y estas acceptance criteria, desglosá la implementación y marcá riesgos". Ese cambio de lenguaje mejora mucho la calidad del resultado.

Anti-patrones

Mezclar todo en un solo archivo

Cuando requirements, spec, task y decisión viven juntos sin estructura, el documento se vuelve imposible de usar. Nadie sabe qué parte es fuente de verdad.

Especificaciones ambiguas

Frases como "debe ser intuitivo" o "debe andar bien" no sirven si no se traducen en criterios verificables. Si no se puede probar, está incompleto.

Tasks demasiado grandes

Una task que dice "implementar todo el flujo" suele terminar en una PR difícil de revisar y en una deuda de trazabilidad.

ADRs a destiempo

Escribir decisiones después de haber olvidado el contexto destruye su valor. Un ADR sirve cuando conserva la discusión real.

Acceptance reescrita para que cierre

Si los criterios de aceptación se ajustan al resultado del código, dejan de ser un contrato y pasan a ser una justificación.

Traces vacías

No alcanza con decir "está en el PR". Si no se puede seguir el hilo hasta el requerimiento original, la trazabilidad no existe.

Runbooks fantasmas

Un runbook que nadie puede ejecutar en una emergencia no sirve. Tiene que ser corto, claro y accionable.

Evaluaciones sin conclusión

Medir algo y no decir qué decisión se tomó es trabajo incompleto. La evaluación tiene que dejar una postura concreta.

Checklist práctico

Antes de cerrar una feature bajo SDD, conviene revisar esto:

- el requerimiento está escrito y tiene problema, objetivo e impacto;
- la spec define comportamiento verificable;
- el alcance excluye lo que no entra;
- las decisiones relevantes están documentadas;
- las tasks están partidas en pasos chicos;
- los criterios de aceptación son concretos;
- hay trazabilidad entre issue, spec, PR y tests;
- existe runbook si el flujo afecta operación;
- quedaron notas de evaluación o validación;
- no hay contradicciones entre documento, código y pruebas;
- el repositorio puede reconstruir el contexto sin depender de memoria oral.

Recomendación de mantenimiento

Para que esta estructura no se vuelva burocrática, conviene sostener tres reglas:

1. escribir poco pero útil;
2. actualizar al mismo ritmo que el código;
3. cerrar cada feature con evidencia.

No se trata de documentar por documentar. Se trata de que la documentación funcione como infraestructura de trabajo. En un repo con IA, esa infraestructura es parte del producto. Reduce errores, mejora la revisión y hace que el conocimiento sobreviva a la rotación de contexto.

Cierre

SDD no es una moda de naming ni un reemplazo mágico de ingeniería. Es una disciplina para ordenar la intención antes de que el código la opaque. En repositorios donde la IA participa en la escritura, la revisión o el análisis, esta disciplina importa todavía más, porque el costo de la ambigüedad se multiplica.

La estructura de [.github/orquestador/sdd](https://github.com/orquestador/sdd) propone algo simple: separar las piezas correctas para que cada una cumpla su función y todas juntas formen un sistema legible. Requirements, specs, decisions, tasks, acceptance

criterios, trases, runbooks, ADRs y evaluation notes no son carpetas decorativas. Son el mecanismo para que una idea pase de problema a implementación sin perder contexto en el camino.

Si el repositorio adopta esta forma de trabajo, cada feature deja de ser un salto de fe. Pasa a ser un proceso rastreable, revisable y mejorable. Y eso, en un stack asistido por IA, vale más que cualquier atajo improvisado.

VOLUMEN 04

AGENTS.md, .github y Comandos Estilo Prompt

Como construir un repositorio preparado para agentes, SDD, Copilot, workflows y prompts reutilizables

Para que existe este tomo

Un repositorio moderno no deberia depender de que el humano recuerde todas las reglas cada vez que abre un asistente de IA. Las reglas del proyecto tienen que vivir en archivos. Algunas reglas son para humanos, otras para agentes, otras para Copilot, otras para CI y otras para el proceso SDD. Cuando esas capas estan mezcladas, la IA trabaja con contexto incompleto y el equipo termina corrigiendo los mismos errores una y otra vez.

Este tomo propone una estructura practica para repositorios que quieren usar IA de forma seria:

1. `AGENTS.md` como contrato operativo para agentes de codigo.
2. `.github/copilot-instructions.md` como instrucciones generales para Copilot.
3. `.github/instructions/*.instructions.md` como instrucciones especificas por path.
4. `.github/prompts/*.prompt.md` como comandos reutilizables estilo prompt.
5. `.github/orquestador/sdd/*` como sistema de especificaciones, decisiones y trazabilidad.
6. `.github/workflows/*` como unica capa ejecutable automatizada.
7. `.github/orquestador/pipelines/catalog.md` como catalogo de gobierno, no como segundo motor de automatizacion.

La idea no es llenar el repo de ceremonias. La idea es que cada archivo tenga un proposito claro y que un agente pueda responder tres preguntas rapido: que proyecto es este, como se trabaja aca y como se verifica que el cambio esta bien.

Mapa de responsabilidades

Archivo o carpeta	Audiencia principal	Responsabilidad
<code>README.md</code>	Personas nuevas	Explicar producto, instalacion y uso
<code>AGENTS.md</code>	Agentes de codigo	Reglas operativas, comandos, permisos, estilo y cierre
<code>.github/copilot-instructions.md</code>	GitHub Copilot	Instrucciones generales persistentes del repositorio
<code>.github/instructions/*.instructions.md</code>	Copilot por zona	Reglas aplicadas a paths concretos
<code>.github/prompts/*.prompt.md</code>	Equipo y asistentes	Prompts invocables para tareas repetidas
<code>.github/orquestador/context/*</code>	Equipo y agentes	Contexto estable: producto, arquitectura, glosario
<code>.github/orquestador/sdd/*</code>	Equipo y agentes	Specs, decisiones, tareas, trazabilidad, runbooks
<code>.github/orquestador/pipelines/catalog.md</code>	Maintainers	Inventario de workflows, permisos, riesgos y owners
<code>.github/workflows/*</code>	GitHub Actions	Automatizacion real: CI, reviewers, validaciones

La regla de oro: un archivo de contexto no debe fingir que ejecuta cosas. Un workflow si ejecuta. Un catalogo documenta. Un prompt orienta. Un `AGENTS.md` gobierna el comportamiento de agentes. Mantener esa separacion evita sistemas confusos.

Estructura recomendada

Una base razonable para un repo que quiere trabajar con IA, SDD y GitHub-first:

```
repo/  
  AGENTS.md  
  README.md  
  src/  
  tests/  
  docs/  
  .github/  
    copilot-instructions.md  
  instructions/  
    frontend.instructions.md  
    backend.instructions.md  
    tests.instructions.md  
  prompts/  
    plan-feature.prompt.md  
    write-spec.prompt.md  
    review-pr.prompt.md  
    generate-tests.prompt.md  
    write-adr.prompt.md  
    qa-harness.prompt.md  
  workflows/  
    ci.yml  
    pr-reviewer.yml  
  orquestador/  
    README.md  
    context/  
      product.md  
      architecture.md  
      glossary.md  
      constraints.md  
    sdd/  
      README.md  
      requirements/  
      specs/  
      decisions/  
      tasks/  
      traces/  
      evals/  
      runbooks/  
    pipelines/  
      catalog.md  
    policies/  
      permissions.md  
      safety.md
```

No todos los proyectos necesitan todo desde el día uno. Pero conviene que la arquitectura mental exista. Un MVP puede empezar con `AGENTS.md`, `context/product.md`, `sdd/specs/`, `prompts/` y `workflows/ci.yml`. El resto se agrega cuando aparece repetición real.

Como escribir un buen **AGENTS.md**

AGENTS.md es el archivo que le dice a un agente como operar en el repositorio. No es una landing page. No es una vision de producto. No es un manual largo para humanos. Es una guia operacional.

Debe responder:

1. Cual es el stack.
2. Donde esta cada cosa importante.
3. Que comandos se usan para instalar, probar, validar y correr.
4. Que archivos o carpetas son delicados.
5. Que estilo de cambios se espera.
6. Que permisos o acciones estan prohibidas.
7. Como cerrar una tarea.

Ejemplo base:

```
# AGENTS.md

## Project Snapshot

Este repo contiene una app Node.js + TypeScript con API en `src/server`,
frontend en `src/web` y tests en `tests`.

## Commands

- Instalar: `npm ci`
- Desarrollo: `npm run dev`
- Tests: `npm test`
- Typecheck: `npm run typecheck`
- Build: `npm run build`

## Working Rules

- Mantener cambios acotados al pedido.
- No modificar migraciones antiguas sin pedir permiso.
- No tocar secretos ni archivos `.env`.
- Preferir tests cerca del comportamiento cambiado.
- Si un comando falla, reportar el error exacto y el proximo paso.

## SDD

- Specs: `.github/orquestador/sdd/specs/`
- Decisiones: `.github/orquestador/sdd/decisions/`
- Tareas: `.github/orquestador/sdd/tasks/`
- Trazabilidad: `.github/orquestador/sdd/traces/`

## Completion

Antes de cerrar, informar:
- Archivos modificados.
- Pruebas ejecutadas.
- Pruebas no ejecutadas y por que.
- Riesgos residuales.
```

El error comun es escribir un `AGENTS.md` demasiado filosofico. Un agente no necesita frases como "usar buenas practicas". Necesita comandos, rutas, limites y criterios.

Jerarquia de instrucciones

Las instrucciones tienen precedencia. Un pedido explicito del usuario tiene mas peso que una regla general del repo. Un `AGENTS.md` mas cercano a una subcarpeta puede especializar reglas del `AGENTS.md` raiz. Las instrucciones de sistema o plataforma tienen prioridad sobre todo lo demas.

La forma practica de pensarlo:

```
Sistema / plataforma
> Usuario en la conversacion
  > AGENTS.md mas cercano al archivo tocado
    > AGENTS.md raiz
      > Documentacion auxiliar
```

Esto sirve especialmente en monorepos. Un root `AGENTS.md` puede decir como se trabaja globalmente, mientras `packages/mobile/AGENTS.md` define comandos y convenciones especificas de mobile.

`.github/copilot-instructions.md`

GitHub documenta las instrucciones personalizadas de repositorio para Copilot como un archivo `.github/copilot-instructions.md`. Su funcion es dar contexto persistente a Copilot cuando trabaja dentro del repositorio.

Este archivo deberia ser mas corto que `AGENTS.md`. No tiene que repetir todos los comandos. Puede enfocarse en estilo, arquitectura, preferencias de respuesta y validaciones esperadas.

Ejemplo:

```
# Copilot Instructions

Este proyecto usa TypeScript estricto. Evitar `any` salvo justificacion.
Preferir funciones puras en la capa de dominio.
No crear dependencias nuevas sin explicar el motivo.
Cuando sugieras codigo, incluir pruebas relevantes.
Si falta contexto de negocio, preguntar o marcar supuesto.
```

Usalo para guiar la calidad general. No lo conviertas en un documento enorme. Si Copilot recibe demasiadas reglas generales, aumenta la probabilidad de conflictos o de que ignore parte del contenido.

`.github/instructions/*.instructions.md`

Las instrucciones por path permiten decir: "cuando trabajes en esta zona del repo, aplica estas reglas". GitHub documenta el patron `NAME.instructions.md` dentro de `.github/instructions`, con frontmatter `applyTo`.

Ejemplo para backend:

```
---  
applyTo: "src/server/**/*.ts,tests/server/**/*.ts"  
---  
  
# Backend Instructions  
  
- Validar entradas en la frontera HTTP.  
- Mantener reglas de negocio fuera de handlers.  
- No acceder a la base de datos desde controllers.  
- Agregar tests de casos borde para errores 4xx y 5xx.
```

Ejemplo para frontend:

```
---  
applyTo: "src/web/**/*.tsx,src/web/**/*.css"  
---  
  
# Frontend Instructions  
  
- Mantener componentes accesibles por teclado.  
- Evitar texto que se desborde en mobile.  
- Usar componentes existentes antes de crear nuevos.  
- No introducir cambios visuales globales sin justificar.
```

La ventaja es precision. En lugar de que una regla de frontend contamine backend, cada zona recibe lo que necesita.

`.github/prompts/*.prompt.md`

Los prompt files son comandos reutilizables. GitHub los documenta como archivos Markdown con extension `.prompt.md`, normalmente dentro de `.github/prompts`. Estan pensados para tareas que el equipo repite: planificar una feature, revisar un PR, generar tests, escribir una ADR, documentar una API o preparar un onboarding.

Conviene tratarlos como comandos versionados. Si un prompt mejora, se revisa en PR. Si falla, se ajusta. Si deja de servir, se borra.

Ejemplo `plan-feature.prompt.md`:

```
---  
description: "Convertir una idea de producto en plan SDD por slices"  
---
```

Usa el contexto de:

- [producto](../orquestador/context/product.md)
- [arquitectura](../orquestador/context/architecture.md)

Entrada del usuario:

`${input:feature:Describe la feature}`

Producir:

1. Problema y objetivo.
2. No objetivos.
3. Supuestos.
4. Requisitos funcionales.
5. Criterios de aceptacion.
6. Slices de implementacion.
7. Tests recomendados.
8. Riesgos y preguntas abiertas.

No inventes reglas de negocio. Marca lo desconocido.

Ejemplo `review-pr.prompt.md`:

```
---  
description: "Revisar un PR con foco en bugs, riesgos y pruebas"  
---
```

Revisa los cambios del PR actual.

Prioridad:

1. Bugs o regresiones.
2. Riesgos de seguridad o permisos.
3. Falta de tests para comportamiento nuevo.
4. Inconsistencias con SDD o ADRs.

Salida:

- Findings con archivo y línea cuando sea posible.
- Preguntas abiertas.
- Resumen breve.

No hagas comentarios de estilo si no afectan mantenimiento o comportamiento.

Ejemplo `qa-harness.prompt.md`:

```
---
description: "Diseñar un harness de evaluacion para una capacidad de IA"
---
```

```
Capacidad a evaluar:
${input:capability:Describe la capacidad}
```

```
Disena:
```

1. Fixtures de entrada.
2. Salidas esperadas o criterios.
3. Rubrica de scoring.
4. Casos negativos.
5. Script CLI minimo.
6. Como integrarlo en CI.
7. Riesgos de falsos positivos.

Estos archivos hacen que el conocimiento del equipo sea invocable. No reemplazan el juicio humano, pero reducen variabilidad.

.github/orquestador

La carpeta `.github/orquestador` funciona como casa del sistema de trabajo. El nombre no es magico. Lo importante es que tenga una responsabilidad clara: reunir contexto, SDD, politicas y catalogos que guian a humanos y agentes.

Una convencion util:

```
.github/orquestador/
  README.md           Indice del sistema operativo del repo
  context/            Contexto estable
  sdd/               Especificaciones y trazabilidad
  prompts/           Prompts internos si no se usan '.github/prompts'
  pipelines/         Catalogo de workflows y automatizaciones
  policies/          Permisos, seguridad, criterios de riesgo
```

Si el equipo usa mucho GitHub Copilot, conviene mantener `.github/prompts` para compatibilidad con herramientas y dejar `.github/orquestador` para contexto y SDD. Si se usa otro agente, se puede tener tambien `orquestador/prompts`. Lo importante es no duplicar sin necesidad.

SDD dentro del repo

SDD significa trabajar desde especificaciones, no desde impulsos. En un repositorio con IA, SDD cumple una funcion critica: evita que el agente implemente una interpretacion creativa del pedido.

Una spec minima deberia incluir:

1. Problema.
2. Objetivo.
3. No objetivos.
4. Requisitos.
5. Criterios de aceptacion.
6. Casos borde.
7. Impacto tecnico.
8. Plan por slices.
9. Pruebas.
10. Trazabilidad con issue, PR y decisiones.

Ejemplo de ruta:

```
.github/orquestador/sdd/specs/2026-05-08-reasignar-reclamo.md
```

Ejemplo de encabezado:

```
# Reasignar reclamo a otro equipo

Estado: Draft
Owner: Producto / Backend
Issue: #123
PRs: pendiente

## Problema

Los operadores no pueden mover un reclamo cuando fue derivado al equipo incorrecto.

## Criterios de aceptacion

- Un operador autorizado puede reasignar el reclamo.
- La reasignacion queda auditada.
- El equipo anterior y el nuevo quedan visibles en el historial.
- Si el reclamo esta cerrado, no puede reasignarse.
```

Cuando el agente implemente, debe poder leer esa spec y saber que significa "terminado".

Workflows como unica capa ejecutable

Si el repositorio usa GitHub como superficie principal, conviene que `.github/workflows` sea la unica capa ejecutable automatizada. Todo lo demas puede documentar, orientar o gobernar. Esta separacion evita dos

problemas: automatizaciones duplicadas y dudas sobre donde ocurre realmente la ejecucion.

Patron conservador para empezar:

```
name: Safe PR Reviewer

on:
  pull_request:
    types: [opened, synchronize, reopened]

permissions:
  contents: read
  pull-requests: read
  issues: write

concurrency:
  group: pr-reviewer-${{ github.event.pull_request.number }}
  cancel-in-progress: true

jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - name: Comment with checklist
        uses: actions/github-script@v7
        with:
          script: |
            const body = [
              "Revision automatica inicial:",
              "- Verificar tests.",
              "- Confirmar criterios SDD.",
              "- Revisar permisos y secretos.",
              "- Mantener cambios acotados."
            ].join("\n");
            await github.rest.issues.createComment({
              owner: context.repo.owner,
              repo: context.repo.repo,
              issue_number: context.payload.pull_request.number,
              body
            });
```

Este workflow no hace checkout, no ejecuta codigo del PR y comenta de forma conservadora. Es una buena primera automatizacion porque aporta orden sin abrir una superficie de riesgo grande.

Catalogo de pipelines

El catalogo no ejecuta. Documenta. Deberia responder:

1. Que workflow existe.
2. Que evento lo dispara.
3. Que permisos usa.
4. Que riesgos tiene.
5. Quien lo mantiene.
6. Que output produce.
7. Que no esta autorizado a hacer.

Ejemplo:

```
# Pipeline Catalog

## safe-pr-reviewer

- Archivo: `.github/workflows/pr-reviewer.yml`
- Evento: `pull_request`
- Permisos: `contents: read`, `pull-requests: read`, `issues: write`
- Ejecuta codigo del PR: no
- Output: comentario con checklist
- Owner: Platform
- Riesgo principal: ruido en PRs si las reglas no se mantienen
- Estado: activo
```

Cuando alguien pregunte "que automatizaciones tiene este repo", el catalogo da la respuesta. Cuando alguien pregunte "que se ejecuta", la respuesta sigue siendo `.github/workflows`.

Como se conectan las piezas

Un flujo completo podria ser:

1. El humano crea un issue.
2. Ejecuta `plan-feature.prompt.md` para convertir la idea en spec.
3. Guarda la spec en `.github/orquestador/sdd/specs/`.
4. El agente lee `AGENTS.md`, contexto y spec.
5. El agente implementa un slice acotado.
6. Corre tests indicados por `AGENTS.md`.
7. Abre PR.
8. `pr-reviewer.yml` comenta checklist.

9. Otro agente o humano usa `review-pr.prompt.md`.

10. Se actualiza trazabilidad en SDD.

Lo valioso es que cada paso deja rastros. El equipo no depende de recordar que se hablo en un chat.

Errores frecuentes

Error	Consecuencia	Correccion
<code>AGENTS.md</code> enorme	El agente ignora partes	Mantenerlo operativo y linkear docs largas
Repetir reglas en cinco archivos	Instrucciones conflictivas	Definir owner por capa
Prompts sin versionar	Cada persona usa variantes	Guardarlos en <code>.github/prompts</code>
Workflows con permisos amplios	Riesgo innecesario	Permisos minimos por workflow
Catalogo que promete ejecutar	Confusion operativa	Catalogo documenta, workflows ejecutan
Specs sin criterios de aceptacion	Implementaciones ambiguas	Cerrar cada spec con pruebas y ejemplos

Checklist de bootstrap

Para dejar un repo listo:

1. Crear `AGENTS.md` con comandos reales.
2. Crear `.github/copilot-instructions.md` breve.
3. Crear `.github/instructions/` solo si hay reglas por path.
4. Crear `.github/prompts/` con 3 a 5 prompts utiles.
5. Crear `.github/orquestador/context/product.md`.
6. Crear `.github/orquestador/context/architecture.md`.
7. Crear `.github/orquestador/sdd/README.md`.
8. Crear `specs/`, `decisions/`, `tasks/`, `traces/`, `evals/`, `runbooks/`.
9. Crear `.github/orquestador/pipelines/catalog.md`.
10. Verificar que `.github/workflows` sea la unica capa que ejecuta automatizacion.
11. Hacer un PR de prueba para confirmar que las instrucciones son entendibles.

Fuentes verificadas

GitHub documenta las instrucciones de repositorio para Copilot, incluyendo `.github/copilot-instructions.md`, `.github/instructions/*.instructions.md` y el uso de `AGENTS.md` para instrucciones de agentes:

- [GitHub Docs - Adding repository custom instructions for GitHub Copilot](#)
- [GitHub Docs - Prompt files](#)
- [GitHub Docs - Your first prompt file](#)
- [openai/agents.md](#)

Nota: los prompt files de Copilot estaban documentados como public preview al verificar estas fuentes el 2026-05-08, por lo que conviene revisar la documentacion oficial antes de imponerlos como estandar rigido en una empresa.

Cierre

La meta no es tener una carpeta `.github` impresionante. La meta es que cada agente que entra al repo pueda trabajar con menos adivinanza, mas verificacion y mejor memoria. `AGENTS.md` da las reglas de operacion. SDD da el contrato de producto y tecnica. Los prompt files convierten tareas repetidas en comandos. Los workflows ejecutan validaciones. El catalogo explica el sistema.

Cuando esas piezas estan alineadas, la IA deja de ser un chat suelto y empieza a ser infraestructura de trabajo.

VOLUMEN 05

Prompt Engineering y Harness Engineering

De prompts sueltos a sistemas versionados, evaluables y productivos

La diferencia entre un experimento útil y un sistema confiable no está solo en escribir un buen prompt. Un prompt aislado puede resolver una tarea puntual, pero un producto serio necesita algo más: versiones, casos de prueba, criterios de evaluación, observabilidad y reglas de seguridad. Ese conjunto es lo que convierte una idea en una capacidad operable.

Este volumen parte de una idea simple: un prompt no debería vivir como una cadena suelta pegada en una app, en un notebook o en un comentario. Si una instrucción importa para el negocio, tiene que poder auditarse, compararse, probarse y desplegarse con disciplina. Ahí entra el harness engineering: el diseño del entorno que ejecuta, mide y controla ese prompt.

La práctica cambia el foco. Ya no se trata de preguntarse “¿qué le digo al modelo?”, sino “¿cómo hago para que esta tarea se ejecute siempre con el mismo criterio, se pueda verificar y no se rompa cuando el sistema crece?”. Esa es la transición de prompt engineering artesanal a ingeniería de prompts productiva.

Qué cambia cuando el prompt entra en producción

En una demo, un prompt puede ser el texto exacto que hoy te dio buen resultado. En producción, ese mismo texto deja de ser suficiente porque aparecen condiciones que antes no importaban:

1. El modelo cambia.
2. La temperatura cambia.
3. Cambia el contexto del usuario.
4. Cambian los datos de entrada.
5. Cambia el equipo que mantiene el sistema.
6. Aparecen requisitos legales, de seguridad o de marca.

Cuando eso pasa, el problema no es solo “calidad de respuesta”. El problema es control. Necesitás saber qué instrucción se usó, con qué versión, sobre qué inputs, con qué herramientas, y si el comportamiento sigue dentro de lo esperado.

Por eso, un sistema de prompts productivo suele tener estas piezas:

- instrucciones persistentes, que describen la identidad y el comportamiento estable;
- prompts de tarea, que resuelven una solicitud concreta;
- fixtures, que representan casos de entrada bien definidos;
- golden tests, que fijan salidas esperadas o propiedades observables;
- evals, que miden calidad con una rúbrica;
- regression tests, que detectan degradaciones;
- reglas de permisos y seguridad;
- logging y observabilidad para revisar resultados en contexto.

No son capas decorativas. Son la infraestructura mínima para confiar en un asistente, un clasificador, un redactor, un router o un agente.

Anatomía de un prompt bueno

Un prompt bueno no es largo ni corto por definición. Es útil cuando reduce ambigüedad, ordena prioridades y deja claro qué hacer cuando faltan datos. En la práctica, los mejores prompts tienen una estructura parecida.

1. Objetivo explícito

El modelo tiene que saber qué problema resuelve. No alcanza con "ayudá al usuario". Conviene especificar el objetivo en términos operativos.

Ejemplo:

```
Tu tarea es transformar pedidos informales en tickets de trabajo claros, accionables y completos.
```

2. Contexto de operación

El prompt tiene que decir en qué entorno se usa y qué límites tiene. No es lo mismo redactar para soporte, para un agente de ventas o para un clasificador interno.

Ejemplo:

```
Este asistente se usa en un equipo de operaciones. Prioriza claridad, trazabilidad y lenguaje profesional.
```

3. Criterios de calidad

Si no definís calidad, el modelo la inventa. Un buen prompt marca lo que se valora: precisión, concisión, cobertura, tono, seguridad, formato.

Ejemplo:

```
La respuesta debe ser concreta, no inventar datos, y separar hechos observables de supuestos.
```

4. Restricciones

Las restricciones evitan respuestas cómodas pero inútiles. Acá entran formato, longitud, herramientas disponibles, idiomas, exclusiones y normas de seguridad.

Ejemplo:

```
No uses tablas si la información es incierta. No asumas permisos. No ejecutes acciones sin confirmación.
```

5. Política de incertidumbre

Un prompt maduro indica qué hacer cuando falta contexto. Eso reduce alucinaciones y respuestas temblorosas.

Ejemplo:

Si falta un dato crítico, hacé una sola pregunta de aclaración. Si el dato no es crítico, seguí con una suposición explícita.

6. Formato de salida

El output debe ser fácil de consumir por humanos o por otra capa del sistema. Si necesitás JSON, marcá el esquema. Si necesitás viñetas, decilo. Si necesitás una plantilla, definila.

Ejemplo:

Devolvé siempre:

1. Resumen
2. Riesgos
3. Próximos pasos

7. Ejemplos

Los ejemplos no son adorno. Son anclas semánticas. Ayudan a fijar tono, nivel de detalle y decisiones límite. En especial sirven cuando la tarea tiene ambigüedad de formato o de criterio.

Un prompt bueno suele mezclar instrucciones con 1 o 2 ejemplos compactos. No hace falta sobrecargarlo: demasiados ejemplos también introducen ruido.

Instrucciones persistentes vs prompts de tarea

Una confusión común es mezclar todo en una sola instrucción. Eso vuelve el sistema frágil. La división útil es esta:

- instrucciones persistentes: lo que no debería cambiar entre tareas;
- prompts de tarea: lo que cambia para cada pedido;
- contexto dinámico: datos del usuario, archivos, estado de la sesión, herramientas, memoria temporal.

Instrucciones persistentes

Son la capa de identidad y política. Definen el rol, el tono, la prioridad de criterios, los límites de seguridad, el idioma y la conducta esperada.

Ejemplo:

Sos un asistente de operaciones. Respondés en español rioplatense neutro, con estilo claro y serio. Priorizás precisión, seguridad y pasos accionables.

Prompts de tarea

Son instrucciones puntuales para una ejecución concreta. Deben ser breves, específicas y enfocadas en el resultado.

Ejemplo:

Con este texto, redactá un resumen ejecutivo de máximo 180 palabras y terminá con 3 riesgos concretos.

Regla práctica

Si una instrucción se repite en todos los casos, probablemente va en la capa persistente. Si cambia por cada ejecución, va en la capa de tarea. Si vive en el input del usuario, no la mezcles con política. Separar estas cosas reduce errores y hace que el sistema sea mantenible.

Fixtures: casos concretos para probar comportamiento

Un fixture es un caso de entrada controlado que representa una situación realista. En prompt engineering, los fixtures son esenciales porque permiten verificar si el sistema responde bien ante escenarios típicos, raros o peligrosos.

No alcanza con un solo ejemplo feliz. Un harness serio necesita fixtures que cubran:

- entradas limpias;
- entradas incompletas;
- entradas contradictorias;
- entradas con ruido;
- pedidos ambiguos;
- intentos de forzar una salida insegura;
- casos límite de formato.

Ejemplo de fixture

```
{
  "id": "ticket-001",
  "input": "Necesito que alguien revise la factura del mes pasado porque creo que vino mal.",
  "expected_traits": [
    "pide aclaracion si falta identificador",
    "no inventa datos",
    "mantiene tono profesional",
    "propone proximo paso"
  ]
}
```

Qué hace bueno a un fixture

Un buen fixture no intenta “ganarle” al modelo con trucos. Describe una situación útil. Tiene que ser:

- estable;
- reproducible;
- legible;

- representativo;
- fácil de ampliar.

Si el fixture cambia todo el tiempo, no sirve para comparar versiones. Si es demasiado artificial, no refleja el uso real. La calidad está en el balance.

Golden tests: fijar comportamiento esperado

Los golden tests son pruebas contra una salida esperada o contra propiedades muy concretas de la salida. Sirven para detectar regresiones cuando cambian el prompt, el modelo o la cadena de herramientas.

Hay dos formas comunes:

Golden exacto

Sirve cuando la salida debe ser muy estable, por ejemplo un formato estructurado.

Ejemplo:

```
{
  "id": "routing-01",
  "expected": {
    "category": "facturacion",
    "confidence": "alta"
  }
}
```

Golden por propiedades

Sirve cuando no querés congelar el texto completo, pero sí el comportamiento.

Ejemplo:

- la respuesta no debe inventar un dato;
- debe incluir una advertencia;
- debe devolver exactamente 3 pasos;
- debe usar el idioma esperado;
- no debe mencionar herramientas no autorizadas.

Este segundo formato es más flexible y suele ser más útil para sistemas con lenguaje natural. El oro no siempre es el string exacto; muchas veces es el cumplimiento de reglas observables.

Harness CLI: correr, comparar, repetir

El harness es el entorno que ejecuta prompts con fixtures, registra resultados y los compara contra una referencia. Una CLI bien pensada hace que todo eso sea repetible desde terminal y CI.

Una estructura razonable separa:

- definición del prompt;
- definición de fixtures;
- configuración del modelo;
- ejecución del lote;
- evaluación;
- reporte;
- exportación de resultados.

Estructura posible

```
prompt-harness/  
  prompts/  
    system.md  
    task.md  
  fixtures/  
    inbox.jsonl  
    safety.jsonl  
    formatting.jsonl  
  evals/  
    rubric.md  
    scoring.ts  
  runs/  
    2026-05-08T10-30-00Z.jsonl  
  reports/  
    latest.md  
  src/  
    cli.ts  
    harness.ts  
    loader.ts  
    evaluator.ts
```

Comandos ejemplo

```
node src/cli.js run --prompt prompts/system.md --fixtures fixtures/inbox.jsonl  
node src/cli.js eval --run runs/latest.jsonl --rubric evals/rubric.md  
node src/cli.js compare --baseline runs/baseline.jsonl --candidate runs/latest.jsonl  
node src/cli.js report --input runs/latest.jsonl --output reports/latest.md
```

Qué debería hacer la CLI

Una CLI útil no solo llama al modelo. También:

- valida que los archivos existan;

- normaliza entradas;
- registra versión de prompt y modelo;
- guarda timestamps;
- serializa respuestas crudas;
- calcula métricas;
- emite un resumen claro para CI.

Si el harness solo imprime texto bonito, es demo. Si también deja trazabilidad y comparación, ya empieza a ser infraestructura.

Evals y rúbricas: medir más allá de “me gusta”

La evaluación de prompts no puede depender solo de intuición. Necesitás criterios repetibles. Ahí entra la rúbrica: una definición explícita de qué significa “bueno”.

Rúbrica simple

Una rúbrica puede puntuar dimensiones como:

- fidelidad al contexto;
- completitud;
- exactitud;
- formato;
- tono;
- seguridad;
- utilidad accionable.

Ejemplo:

```
Puntaje 0-2 por dimensión:  
0 = falla grave  
1 = parcial  
2 = cumple
```

Ejemplo de evaluación

Dimensión	Criterio
Fidelidad	No inventa datos ni contradice el input
Formato	Respetar la estructura pedida
Tono	Mantiene lenguaje profesional
Accionabilidad	Da próximos pasos útiles
Seguridad	No ejecuta ni recomienda acciones prohibidas

Evaluación automática y humana

Conviene combinar ambos enfoques.

- automática: para reglas objetivas, formato, longitud, presencia de campos, patrones prohibidos;
- humana: para matices de calidad, claridad, persuasión, utilidad y alineación.

En sistemas reales, el error más común es usar métricas automáticas pobres como si resolvieran todo. Sirven, pero no reemplazan la lectura crítica. La buena práctica es usar automatización para filtrar y rúbrica humana para decidir.

Regression tests: que no se rompa lo que ya andaba

Un regression test compara comportamiento entre versiones. El objetivo no es congelar el sistema para siempre, sino detectar cambios no deseados.

En un flujo maduro, cada cambio de prompt o de modelo debería responder:

1. qué mejoró;
2. qué empeoró;
3. qué casos nuevos aparecieron;
4. qué se acepta como trade-off;
5. qué necesita rollback.

Casos de regresión típicos

- el prompt nuevo empieza a ser más verboso de lo necesario;
- desaparece una advertencia de seguridad;
- el modelo deja de pedir aclaraciones;
- cambia el idioma;
- el formato JSON rompe un parser;
- una herramienta se invoca cuando no debería.

Estrategia práctica

Mantené un set pequeño de fixtures críticos y un set más amplio de fixtures exploratorios. Los críticos protegen lo esencial. Los exploratorios te muestran cómo se comporta el sistema fuera del camino principal.

Si una regression test falla, no basta con "arreglar el output". Hay que entender si el problema está en:

- el prompt;
- el modelo;
- el postprocesado;
- la configuración;
- la política;
- el set de datos.

Seguridad y permisos

Cuando un modelo produce acciones, no alcanza con que responda bien. También tiene que actuar dentro de límites claros. En sistemas con herramientas, la seguridad forma parte del diseño del prompt y del harness.

Principios básicos

- mínimo privilegio;
- confirmación para acciones sensibles;
- separación entre sugerir y ejecutar;
- validación de input y output;
- logging de decisiones;
- bloqueo explícito de acciones peligrosas.

Ejemplo de regla

```
Si una acción modifica datos, cobra dinero, borra información o envía mensajes externos, pedir confirmación humana antes de ejecutarla.
```

Prompt seguro

El prompt no debe dar por sentados permisos ni credenciales. Tampoco debería alentar al modelo a "resolver por su cuenta" cosas que requieren validación externa.

Ejemplo:

```
No asumas que tenés acceso a sistemas externos. Si hace falta una acción con impacto, describila y pedí confirmación.
```

Harness seguro

El harness debe poder simular permisos y probar límites:

- sin acceso a internet;
- con herramientas deshabilitadas;
- con credenciales falsas;
- con modo solo lectura;
- con confirmación requerida.

Eso permite verificar que el sistema no solo funciona cuando todo está habilitado, sino también cuando opera bajo restricciones.

Observabilidad: ver qué pasó de verdad

La observabilidad es lo que permite depurar y aprender del sistema una vez que salió del laboratorio. Si un prompt falla en producción, necesitás reconstruir el contexto.

Qué conviene registrar

- versión del prompt;
- versión del modelo;
- fecha y hora;
- entrada resumida;
- salida cruda;
- herramientas usadas;
- latencia;
- costo estimado;
- errores;
- decisión de policy;
- identificador del fixture o caso.

Ejemplo de log

```
{
  "run_id": "2026-05-08T10:30:00Z",
  "prompt_version": "1.4.2",
  "model": "gpt-5.3",
  "fixture_id": "safety-03",
  "latency_ms": 1840,
  "tools_used": ["search"],
  "outcome": "needs_review"
}
```

Qué mirar primero

Cuando algo falla, conviene revisar:

1. el input original;
2. el prompt real aplicado;
3. la configuración del modelo;
4. los tools disponibles;
5. la salida cruda;
6. el postprocesado;
7. el criterio de evaluación.

Sin observabilidad, cada bug parece magia. Con observabilidad, se vuelve una secuencia de decisiones auditables.

Un ejemplo mínimo de harness

Para aterrizar la idea, imaginá un sistema que clasifica solicitudes internas y responde con un plan breve. La arquitectura mínima puede verse así:

```
1. cargar instrucciones persistentes
2. cargar prompt de tarea
3. cargar fixture
4. construir mensaje final
5. ejecutar modelo
6. validar formato
7. puntuar con rúbrica
8. guardar resultado
9. comparar contra baseline
10. reportar
```

Pseudocódigo

```
const system = loadFile("prompts/system.md")
const task = loadFile("prompts/task.md")
const fixtures = loadJsonl("fixtures/inbox.jsonl")

for (const fixture of fixtures) {
  const input = buildInput(system, task, fixture)
  const output = await model.run(input)
  const score = evaluate(output, fixture.expected_traits)
  saveRun({ fixtureId: fixture.id, output, score })
}
```

Lo importante del ejemplo

No hace falta sofisticación para empezar. Lo esencial es que el flujo sea:

- explícito;
- reproducible;
- versionado;
- evaluable;
- comparable.

Una vez que eso existe, se puede escalar. Antes de eso, todo es difícil de mantener.

Criterios para iterar prompts sin romper el sistema

Cuando mejorás un prompt, no conviene editarlo a ciegas. Es mejor seguir un ciclo corto y disciplinado.

Checklist de iteración

- definir el problema exacto;
- elegir un fixture representativo;
- escribir la hipótesis de mejora;
- cambiar una sola cosa por vez;
- correr el harness;
- comparar contra baseline;
- revisar fallos nuevos;
- aceptar el cambio o revertirlo;
- documentar la decisión.

Reglas prácticas

1. No mezcles mejoras de formato con cambios de política en la misma iteración.
2. No uses un solo ejemplo feliz para justificar un cambio.
3. No declares victoria sin mirar regresiones.
4. No guardes un prompt nuevo sin su versión y su motivo.
5. No metas lógica de negocio crítica solo dentro del prompt si puede vivir mejor en código.

Cuándo conviene mover lógica fuera del prompt

Un prompt no debería cargar con todo. Si una regla es estricta, verificable y central al negocio, muchas veces conviene codificarla fuera del modelo.

Ejemplos:

- validación de esquema;
- reglas de permisos;

- enrutamiento determinístico;
- filtrado de datos sensibles;
- normalización de formatos;
- cálculo de métricas.

El prompt queda para lo que hace mejor: interpretar, priorizar, redactar, sintetizar y decidir con contexto. El código queda para lo que hace mejor: validar, controlar y ejecutar de forma determinista.

Señales de madurez

Sabés que pasaste de “prompt suelto” a “sistema” cuando podés responder estas preguntas sin improvisar:

- ¿qué versión del prompt está en producción?
- ¿con qué fixtures se valida?
- ¿qué criterios define la rúbrica?
- ¿qué cambió respecto del baseline?
- ¿qué acciones están permitidas?
- ¿qué quedó registrado en los logs?
- ¿qué regresiones se toleran y cuáles no?

Si esas respuestas están dispersas, el sistema todavía depende demasiado de memoria humana.

Resumen operativo

Prompt engineering no es solo redactar mejor. Es diseñar una interfaz de instrucción que se pueda sostener en el tiempo. Harness engineering es el soporte que hace posible esa sostenibilidad: carga fixtures, ejecuta versiones, evalúa resultados, registra evidencia y protege el sistema de cambios accidentales.

La disciplina práctica queda en una fórmula simple:

- separar instrucciones persistentes de prompts de tarea;
- usar fixtures reales y representativos;
- fijar expectativas con golden tests y rúbricas;
- correr regression tests antes de publicar cambios;
- controlar permisos y acciones sensibles;
- observar lo que pasa en producción;
- versionar todo lo que importe.

Cuando eso existe, el prompt deja de ser una apuesta. Se vuelve una pieza de ingeniería.

Checklist final

- ☐ Definir la instrucción persistente del sistema.

- ☐ Separar el prompt de tarea del contexto dinámico.
- ☐ Crear fixtures para casos felices, ambiguos y peligrosos.
- ☐ Escribir al menos un golden test por comportamiento crítico.
- ☐ Diseñar una rúbrica simple y repetible.
- ☐ Implementar un harness CLI con `run`, `eval` y `report`.
- ☐ Guardar versionado de prompts, modelo y configuración.
- ☐ Registrar logs y latencia por ejecución.
- ☐ Probar permisos, fallbacks y casos sin herramientas.
- ☐ Comparar cada cambio contra un baseline.
- ☐ Documentar decisiones de cambio y rollback.

Ese conjunto alcanza para pasar de una idea prometedora a una capacidad productiva, auditable y mantenible.